

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM

Khoa Công Nghệ Điện Tử - Bộ môn Điện Tử Máy Tính

BÀI GIẢNG

**HỆ THỐNG NHÚNG
(Embedded System)**

TP.HCM-2024

NỘI DUNG MÔN HỌC

Chương 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Chương 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

Chương 3: KỸ THUẬT PHẦN MỀM

Chương 4: THIẾT KẾ HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1. **NGÔN NGỮ LẬP TRÌNH**
2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

1. NGÔN NGỮ LẬP TRÌNH

Contents

- ☐ Requirements
- ☐ Specification languages

1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

+ **Hierarchy**

- Behavioral and structural.

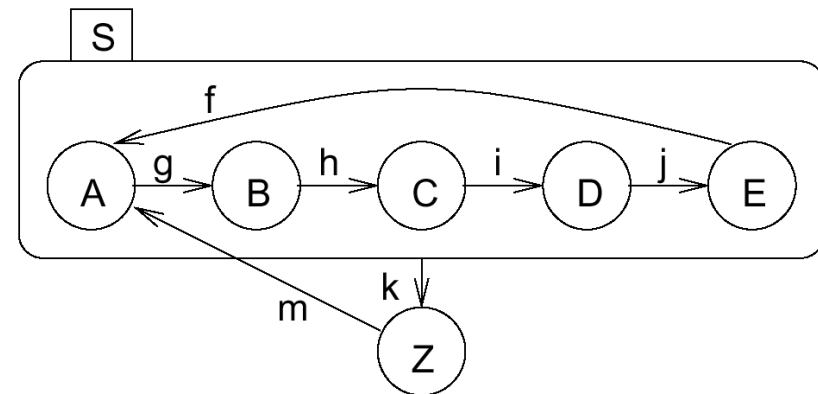
+ **Timing behavior**

+ **Compositional behavior**

- Must be “easy” to derive behavior from behavior of subsystems.

+ **State-oriented behavior**

- Classical automatic models are insufficient.
- Required for reactive systems.



1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

+ **Event-handling**

- **External** (caused by the environment) or **internal events** (caused by the system) .

+ **No obstacles to the generation of efficient implementations**

+ **Support for the design of dependable systems**

- **Unambiguous semantics** and capable of describing **security** and **safety requirements**.

+ **Synchronization and communication**

- Concurrent actions have to be able to communicate

1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

+ **Concurrency** → offers several **distinguished advantages** to programmers and application users.

- The system appears to **immediately respond** to every user request
- It improves the **processor utilization**
- It provides a convenient structure for **failure isolation**

+ **Verifiability**

+ **Readability** → Specification must be readable by the human being.

+ **Executability**

- The possibility to execute a specification is a way for checking it.

1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

+ **Presence of programming elements**

- Programming languages have proven to be convenient for the expression of computations.
- Classical state diagrams do not meet the requirement.

+ **Support for the design of large systems**

- Software technologies has found object orientation mechanism for designing large systems

1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

- + **Portability and flexibility**

- **Small changes** of the system's features should **require small changes** to the specification.

- + **Non-functional properties**

- **Fault tolerance**, size, **expected lifetime**, **power consumption**, electromagnetic compatibility, **extendibility** etc.

- + **Support for non-standard IO-devices**

- + **Appropriate model of computation**

1. NGÔN NGỮ LẬP TRÌNH

1.1 Requirements

+ Specifying timing → Four requirements:

- Access to a timer – a means to **measure** **elapsed time**.
- Means for **delaying** a **process** (e.g. “wait for” in VHDL).
- Possibility to **specify** **timeouts** (e.g. StateCharts allows timeouts).
- Methods for **specifying** **deadlines** and **schedules**.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- Statemate system
- SDL
- SystemC
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

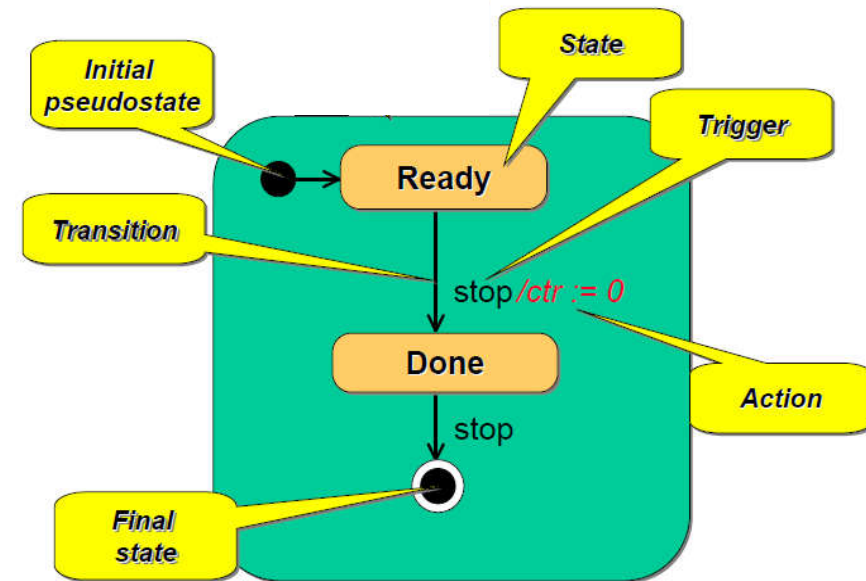
❑ State Transition Diagrams (STD) / Finite State Machine (FSM)

- Help understand the **behavioral aspect** of the system.

- Simple directed graphs with **nodes** denoting states & **arrows** denoting transitions.

- **State:** a combination of different input values for a particular moment of time.

- **Transitions:** Change of state depending on the input received.



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

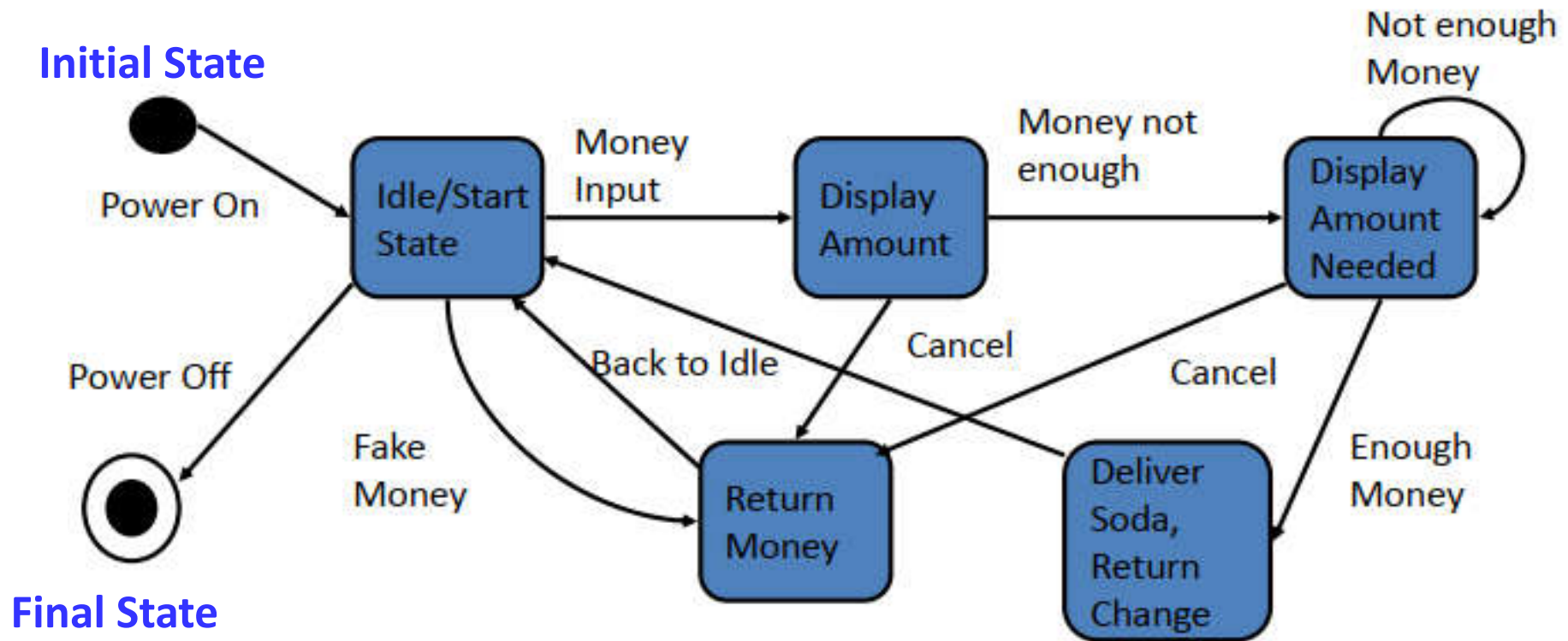
□ Finite State Machine (FSM)

- **FSM** = (Inputs, Outputs, States, InitialState, NextState)
- Often suitable for controllers, protocols etc.
- Rarely suitable for Memory and Datapaths.
- Easy to use with powerful algorithms for synthesis and verification.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

❑ Finite State Machine (FSM) → Example: Vending machine



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

❑ Limitation of FSM

- **Scalability** - Number of states & transitions **increase exponentially** as the system complexity increases.
 - No **concurrency** support.
 - Have an **unstructured**, **unrealistic**, and **chaotic** state diagram.
- To address these problems Harel proposed **StateCharts** → This is an extension of State Transition Diagram / Finite State Machines.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- **Harel's StateCharts**
- UML Statecharts
- Statemate system
- SDL
- SystemC
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.1 Harel's StateCharts

- + Introduced by David Harel in 1987.
- Provide **compact** and **expressive visual formalisms** for reactive systems.
- + **What are Statecharts?**
 - Viewed as both **model** and **graphical** specification language.
 - Describe communicating **Finite State Machines (FSMs)**
 - **Visual formalism** for describing **states & transitions** in **modular fashion**.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

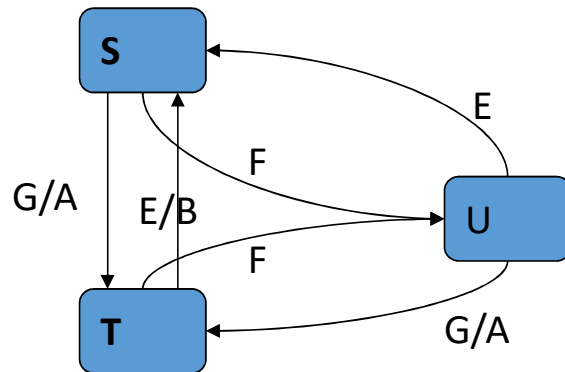
1.2.1 Harel's StateCharts

- + The **Harel statechart** is equivalent to a **State Diagram** but it **improves** the **readability** of the resulting diagram.
- + Statecharts based on a **generalization** of the concepts of **FSMs**.
 - *It's considered that* **the computing power** of Statecharts is the same as that of Finite State Machines.
- + Used as a **modeling tool** and adopted by **Unified Modeling Language (UML)** as **an important technique to model** the dynamic behavior of systems.

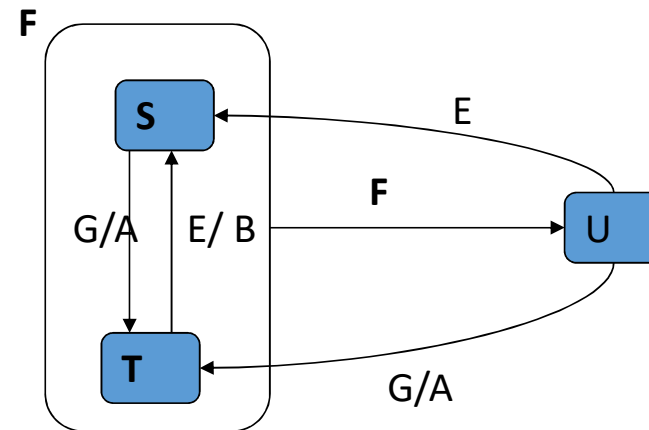
1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.1 Harel's StateCharts



State-Transition Diagram



Statechart

- The **statechart** **reduces** the number of **transitions** when compared to **State Transition Diagrams**.
- **G/A** represents that for **event G**, **action A** takes place.
- **S & T** are combined together into a **super-state F** (**OR-property** of the statechart)¹⁹

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.1 Harel's StateCharts

- + **StateCharts**' main application domain is that of **local, control-dominated systems**.
- + **Key advantage** is the property of nesting hierarchies (Nested States).
 - Allows states to be **nested within other states**, reducing complexity.
 - **Parent states** contain **substates**, reducing redundancy in transitions.
- + Examples of **tools** based on **StateCharts**:
 - **StateMate** (*in the next section*), StateFlow, BetterState, ...

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

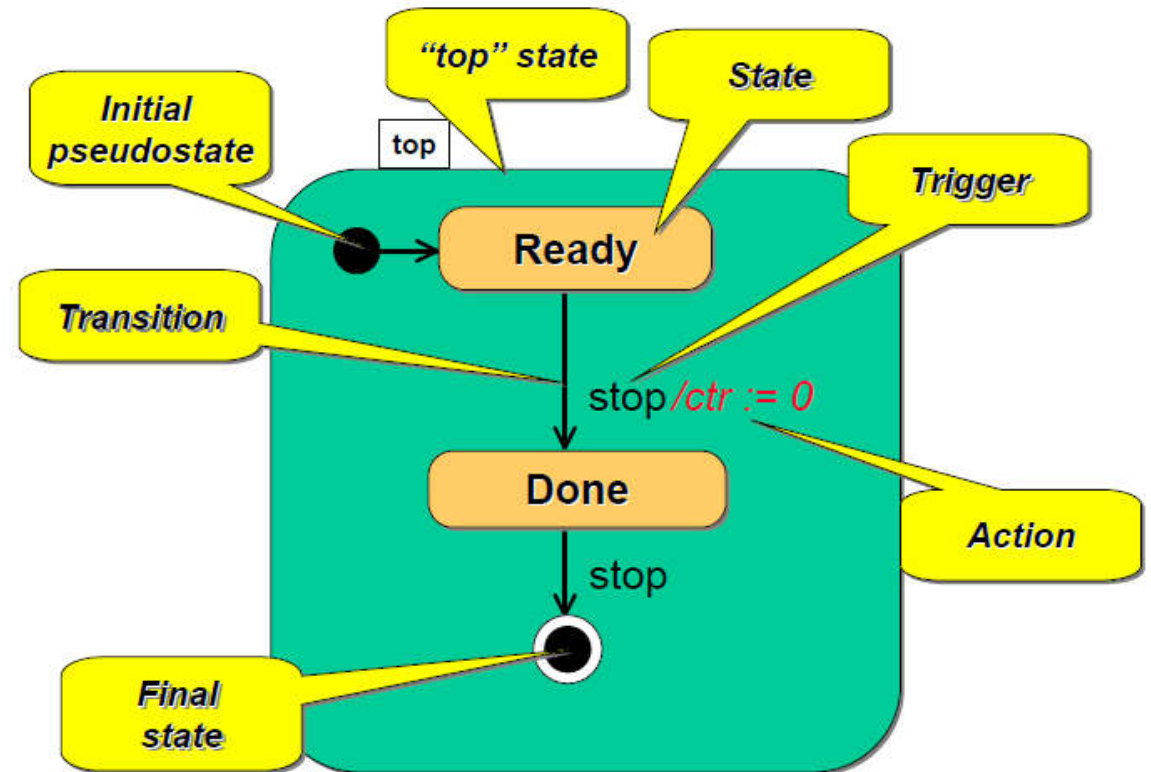
- Harel's StateCharts
- **UML Statecharts**
- Statemate system
- SDL
- SystemC
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.2 UML StateCharts

→ Basic UML (Unified Modeling Language)

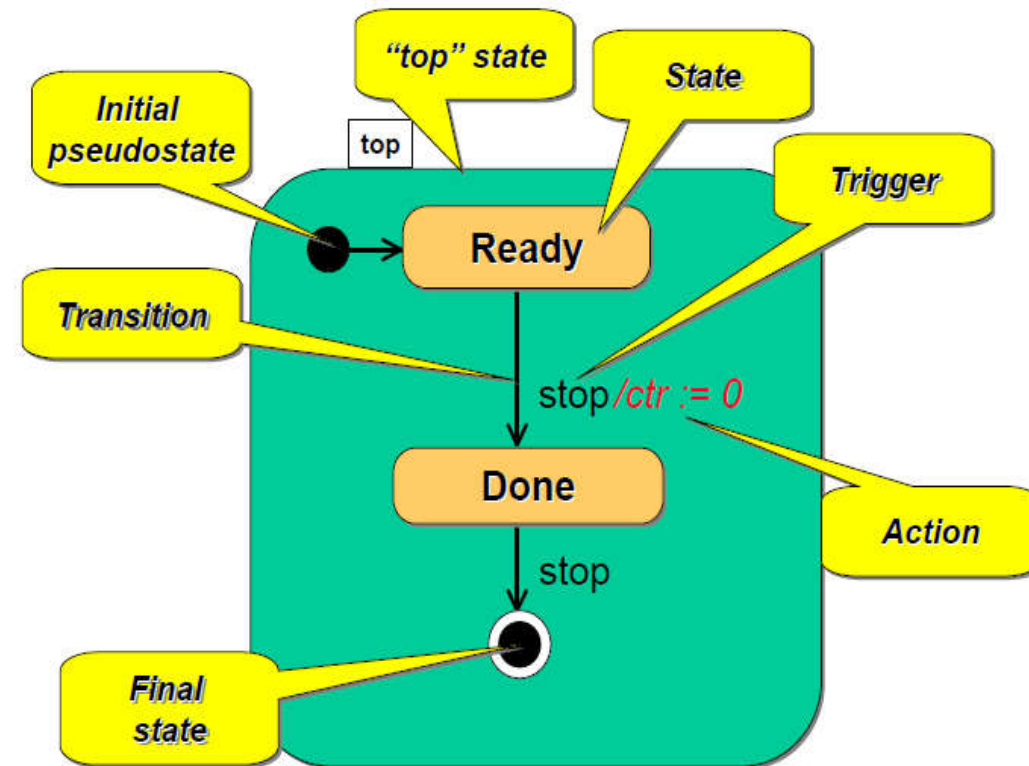


1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.2 UML StateCharts

- + Harel statecharts are the **basis** for UML statecharts.
- + Harel statecharts were mainly designed for **function-oriented structured** analysis design techniques.
- + Statecharts were introduced in UML with **modifications** in the **semantics** and **some in-build terminology**.



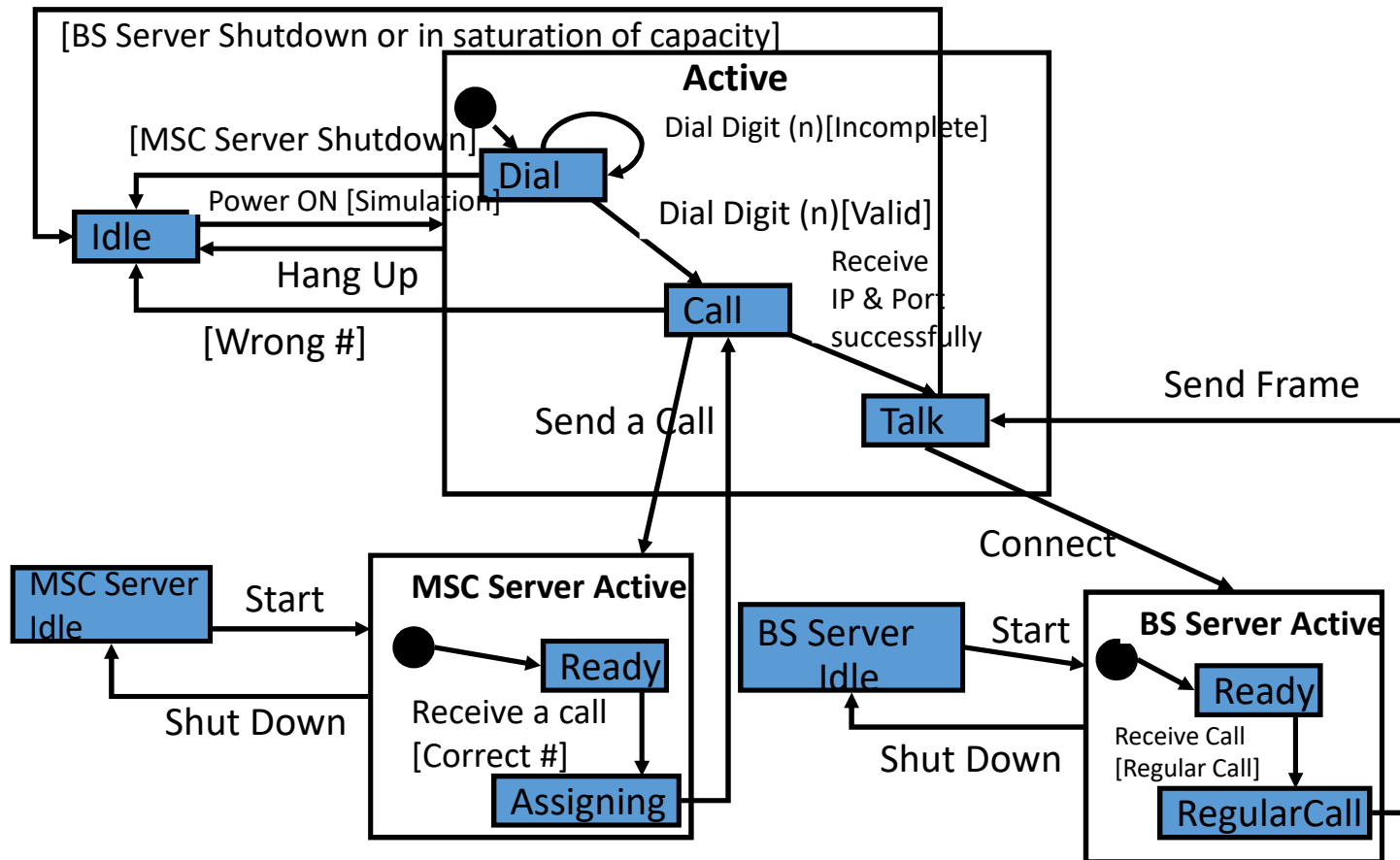
1. NGÔN NGỮ LẬP TRÌNH

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1.2 Specification languages

1.2.2 UML StateCharts

Example:
Mobile Phone System

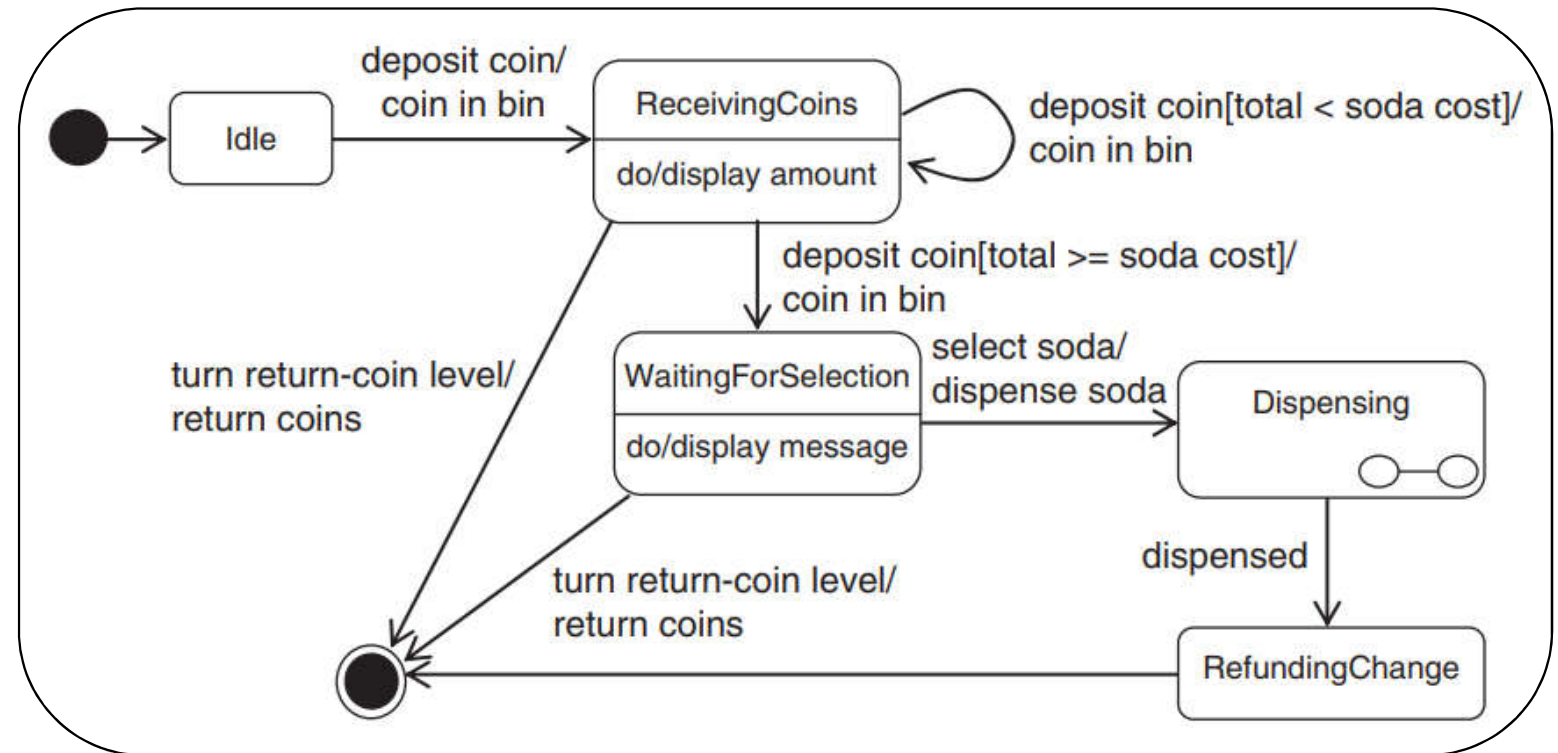


1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.2 UML StateCharts

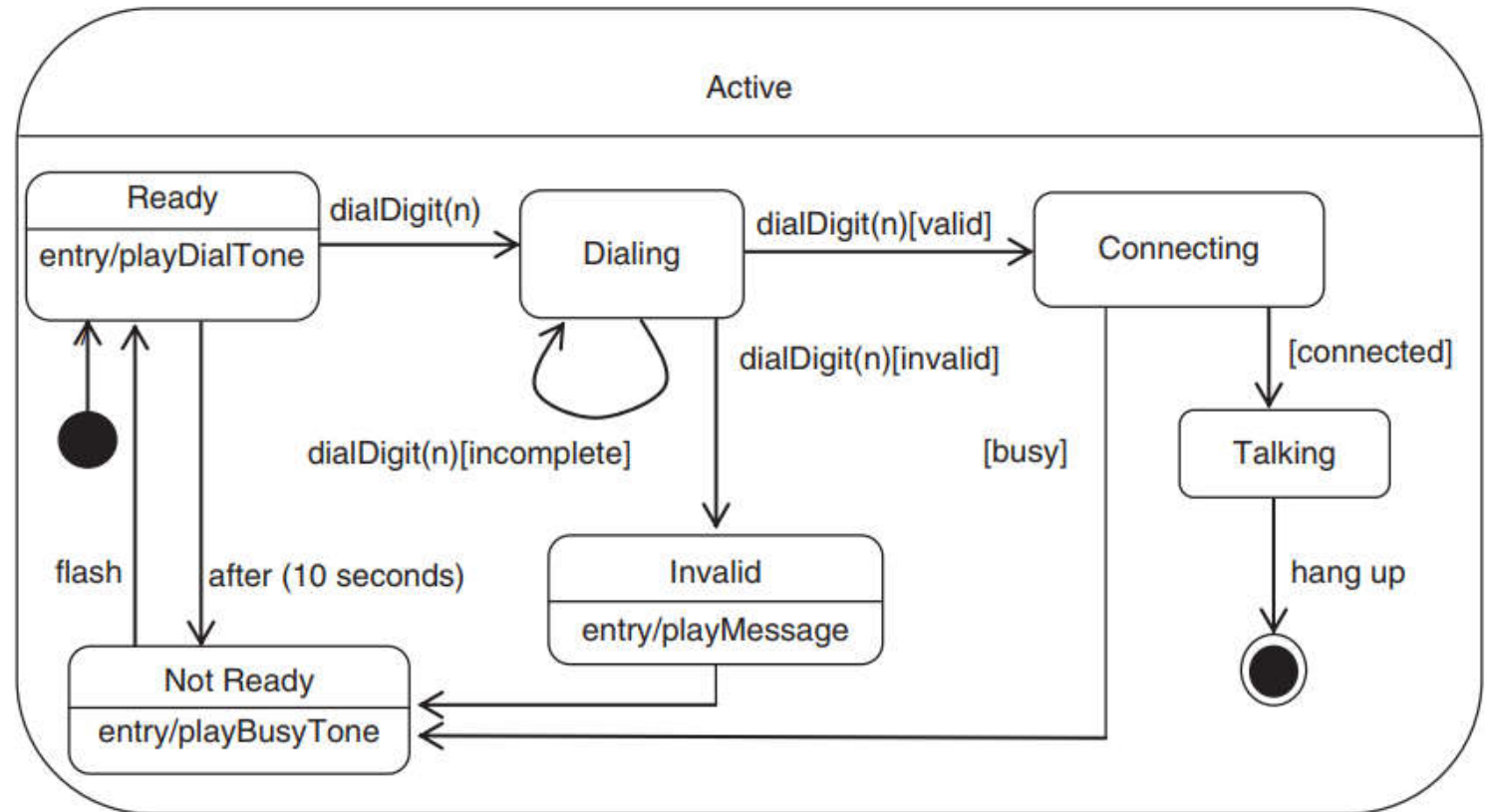
Example: Soda Machine



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.2 UML StateCharts A partial representation of composite state of **Phone**.



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.2 UML StateCharts

- + Use of **Harel statecharts** or **UML statecharts** depends on the requirements for the design of the system.
- + Systems which have **synchronization as their main issue** can be **well-designed if they used UML statecharts**.
- + **UML statecharts** are widely used **due to popularity**, and **number of tools** (typically is **statemate**) in market that support **UML statecharts**.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- **Statemate system**
- SDL
- SystemC
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 Statemate system

- + **A graphical working environment** developed by Harel to **address** the **semantic shortcomings** of **statecharts**.
- + **Enables user** to prepare, analyze, and debug **diagrammatical description** of the system under development.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 Statemate system

- + **It provides** a direct and structured approach between user requirements and software implementation.
- + **Creates** a visual, graphical specification that clearly and precisely represents the intended functions and behavior of the system.
- + **This specification** may be executed, or graphically simulated.
- + **Uses:**
 - Military and Aerospace
 - Automotive Manufactures and Suppliers
 - Medical Electronics
 - Railway Systems

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 Statemate system

- **Model-based approach**
 - Enables a model-based approach allowing **errors** to **be detected and corrected earlier** in the process.
 - Enabling the system engineer to **create formal requirements** and an **executable specification** while generating system, integration and unit tests.
- **Modeling languages**
 - The modeling language is based on **standard engineering diagrams**.
 - The **three views** of **the system model** are described:
 - 1) **Structure: Module-charts;**
 - 2) **Functionality: Activity-charts;**
 - 3) **Behavior: Statecharts**

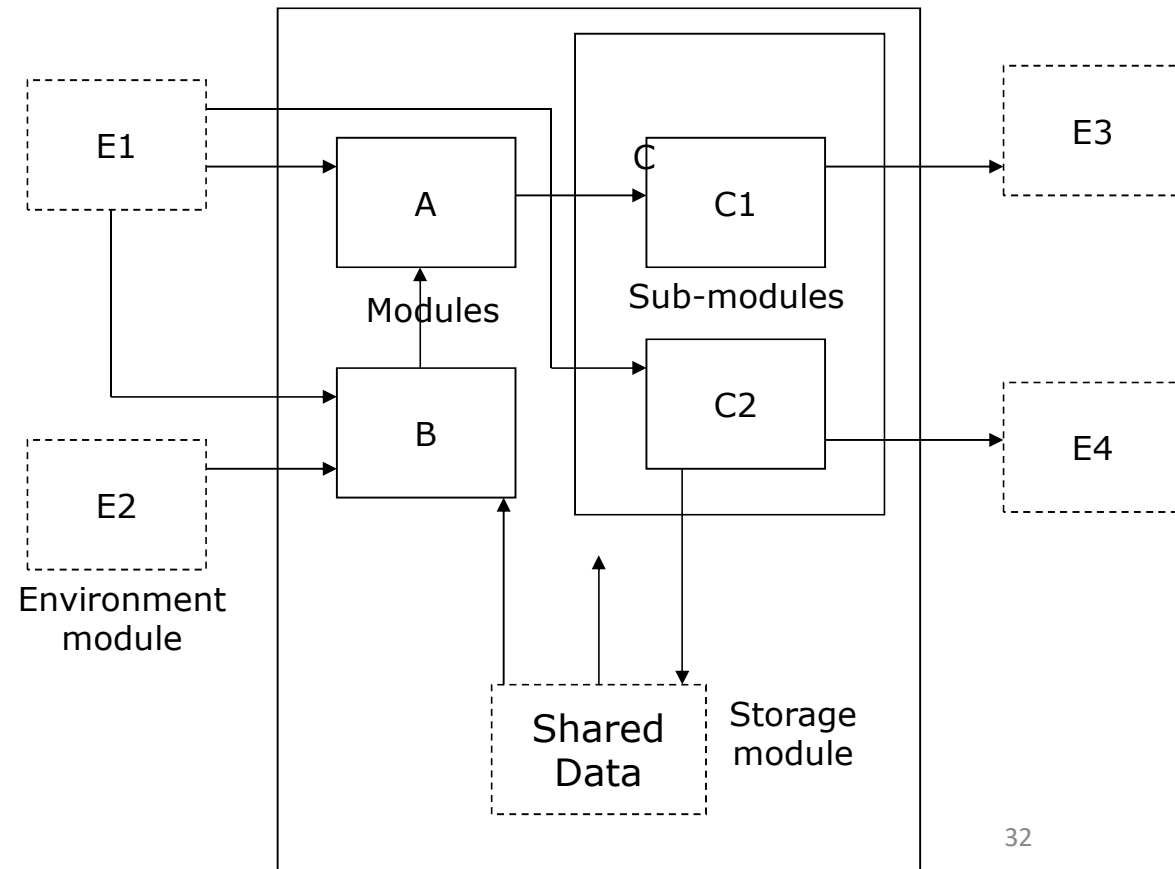
1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 Statemate system

1) Module-charts (Structure)

- Can be regarded as a certain kind of **data-flow diagram**.
- Describe the modules by its division into **hardware and software blocks** and **their inner components**, and **the communication** between them.



1. NGÔN NGỮ LẬP TRÌNH

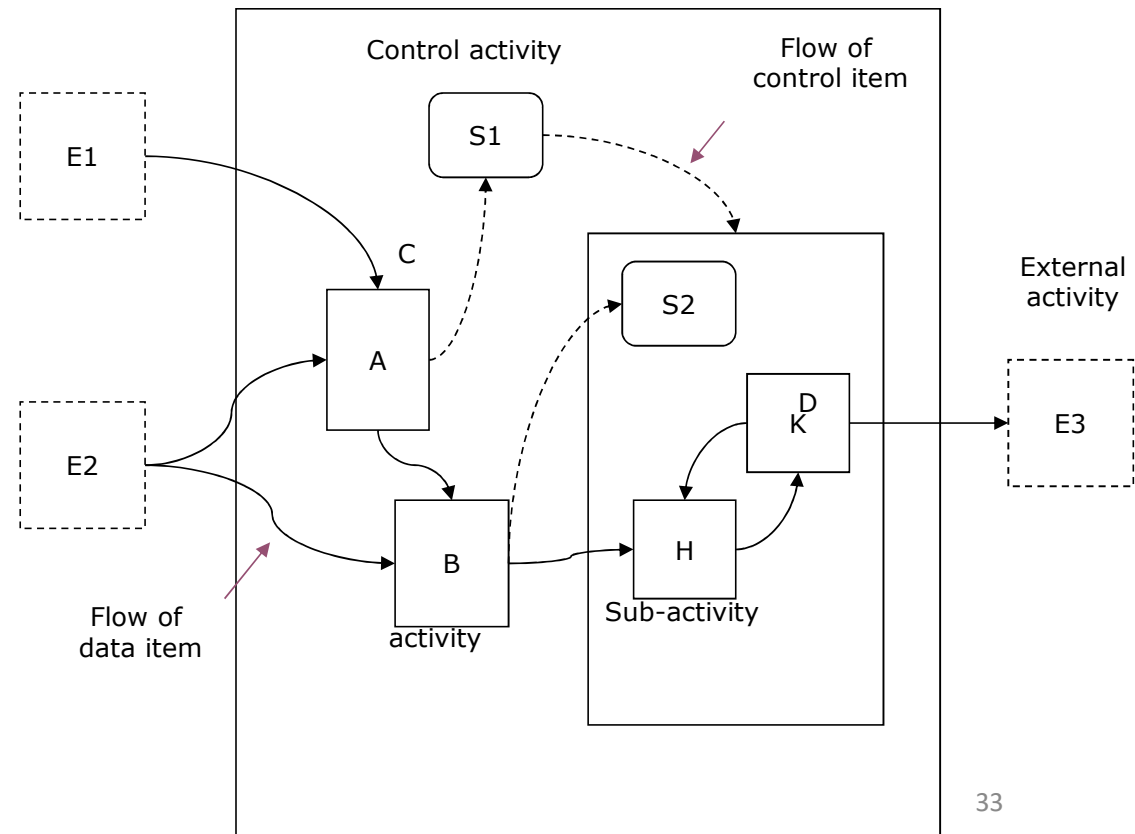
1.2 Specification languages

1.2.3 State machine system

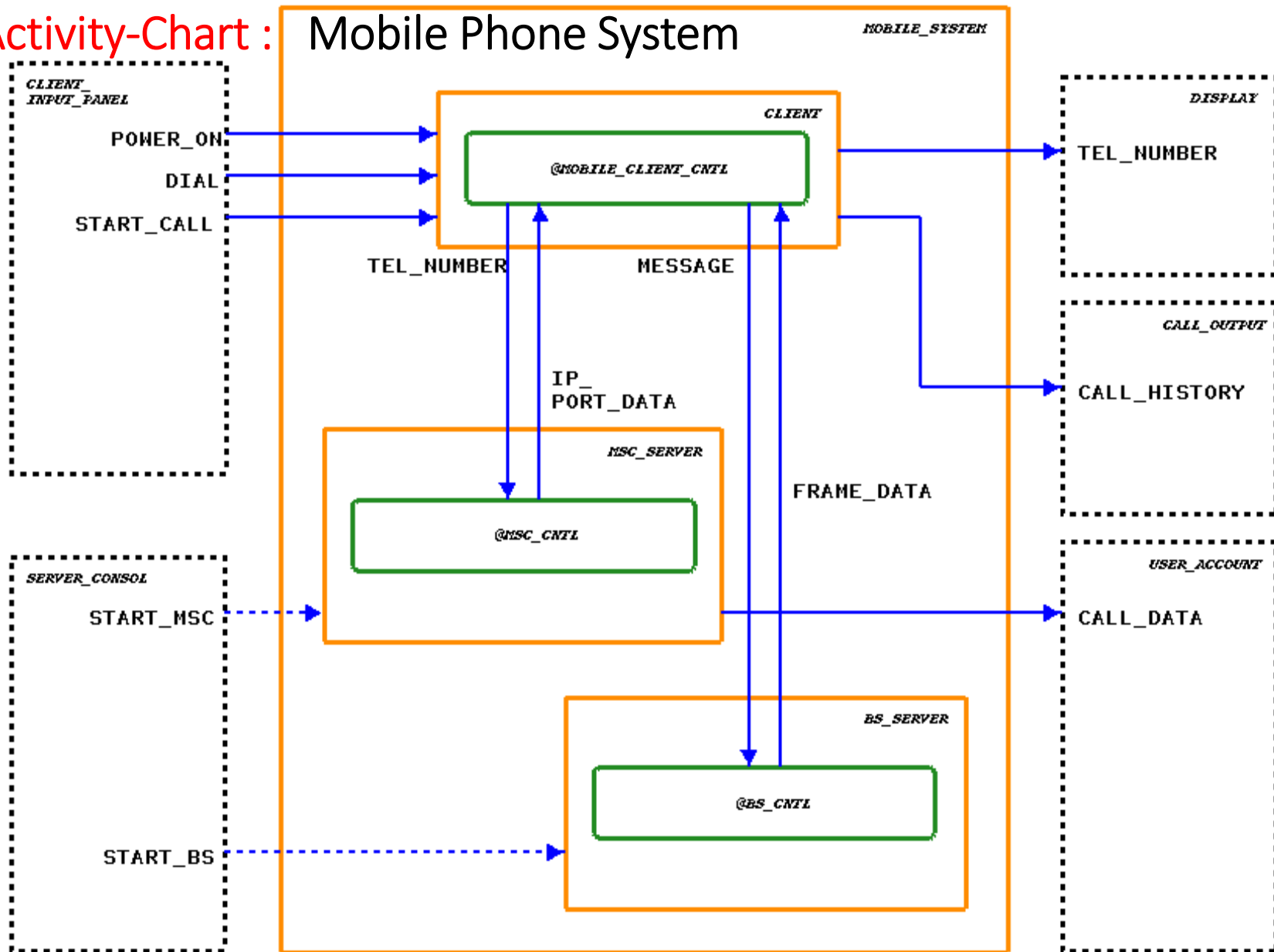
2) Activity-charts (Functionality)

- Activity-charts can be viewed as **multi-level data-flow diagrams**.
- **Capture** functions, or activities, as well as data-stores, all organized into hierarchies and connected via the information that flows between them.

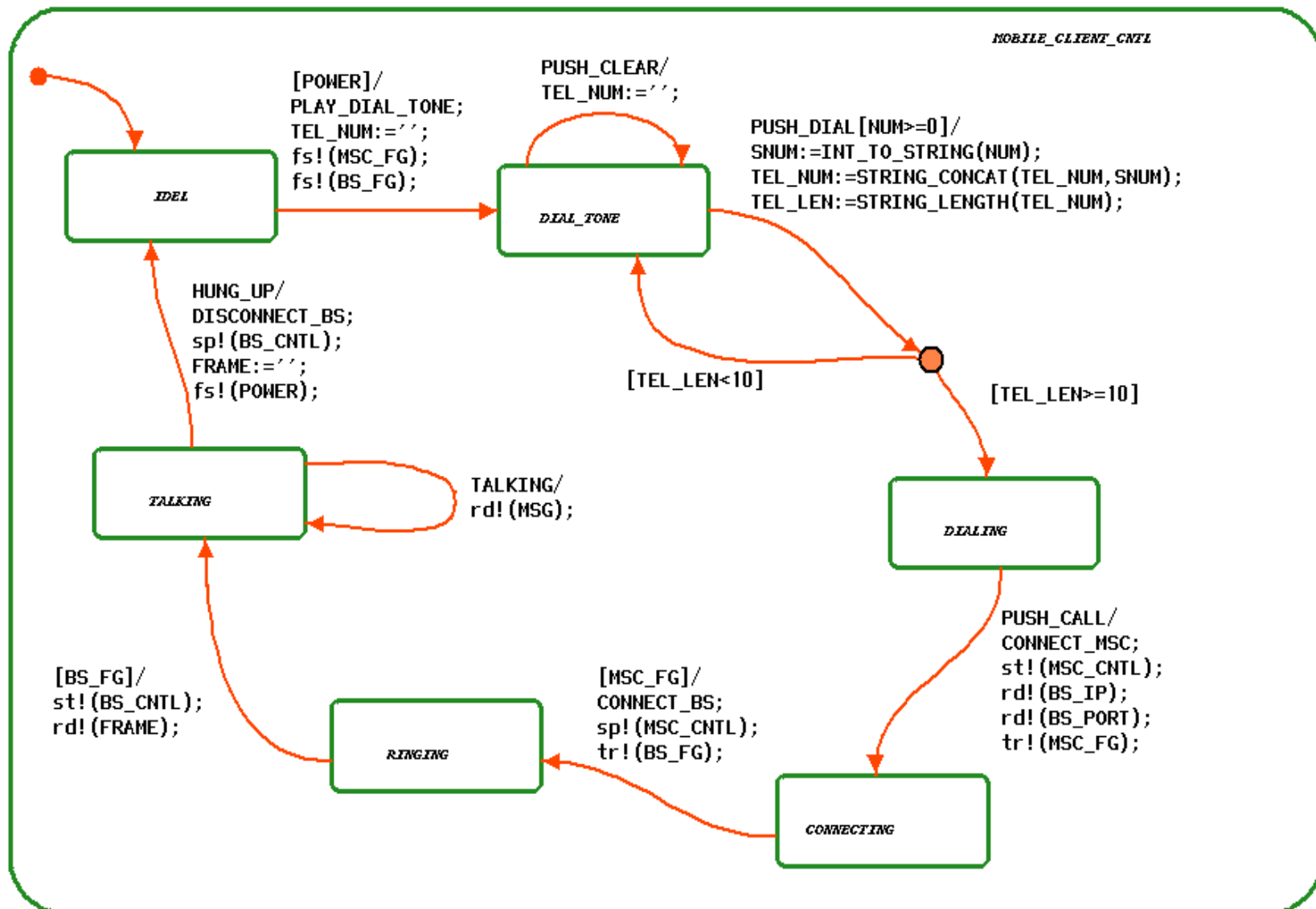
(Figure: The internal description)



Activity-Chart : Mobile Phone System



Statechart (behavior) of Client: Mobile Phone System



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 State machine system (Simulation)

- + **State machine model** is a formal model that can be **simulated** and automatically **translated into code**.
- + **System behavior** is **validated** as an integral part of the design process before anything is built.
- + **State machine simulator** can **provide** traditional debugging: monitors, and debugger windows.
 - Ensure that its **behavior is correct** and to **capture the test data** that **will be used later** to test the implementation.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.3 Statemate system

- + The Statemate system can generate **high quality C code** for software developer, and **VHDL/Verilog code** for hardware engineers.
- + The software creates a **virtual prototype** for operation on a workstation or PC, or **code** that runs on the target test bench system.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- Statemate system
- **SDL**
- SystemC
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

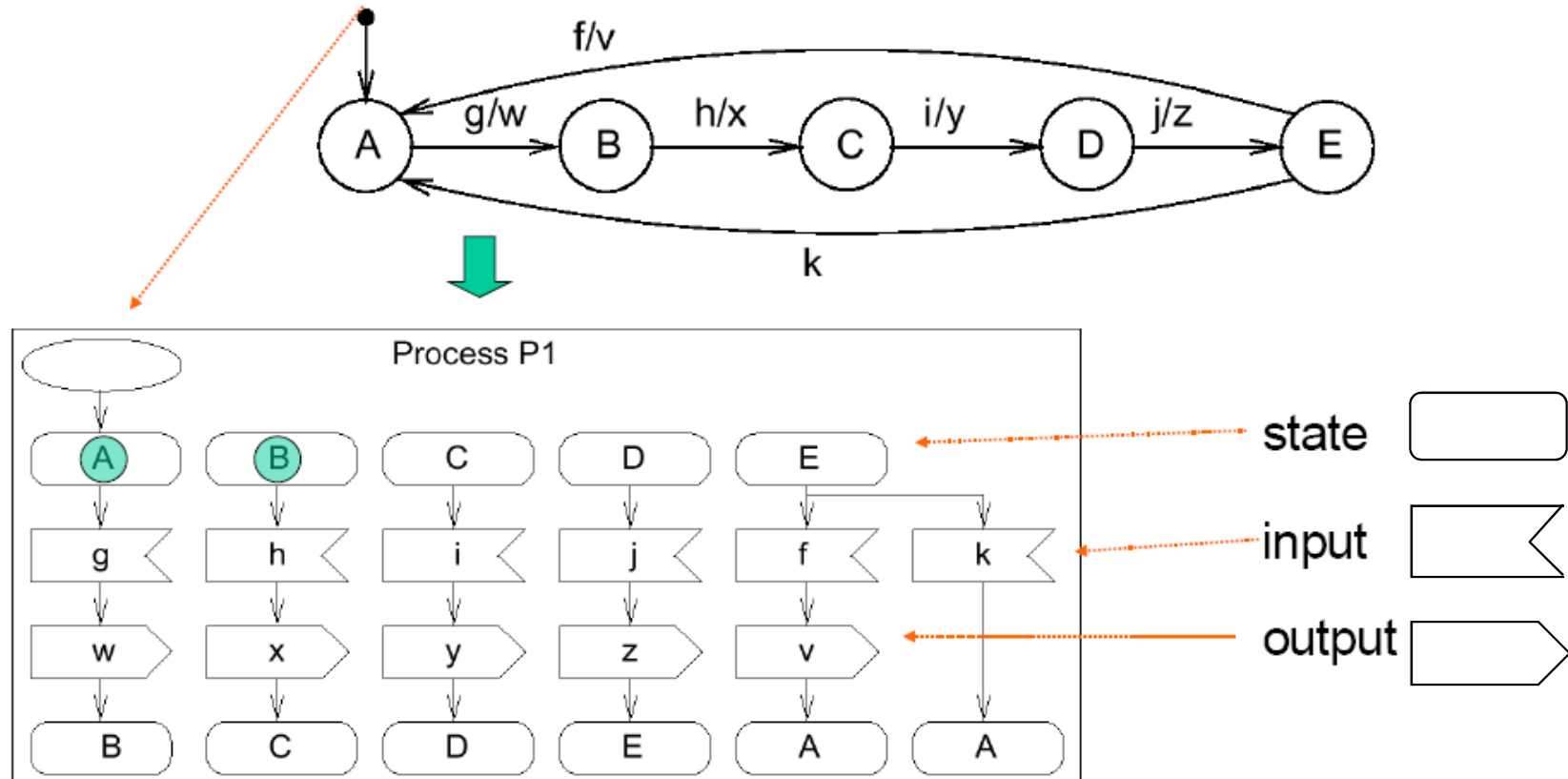
1.2.4 SDL (Specification and Description Language)

- + Designed to model Distributed systems.
- + Based on asynchronous message passing.
- + Provides both Textual as well as Graphical.
- + Processes (represent extended FSMs) are the basic elements.
- + Processes can perform operations on data.
- + Contains programming language elements such as procedures.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.4 SDL → An representation of FSMs (processes)

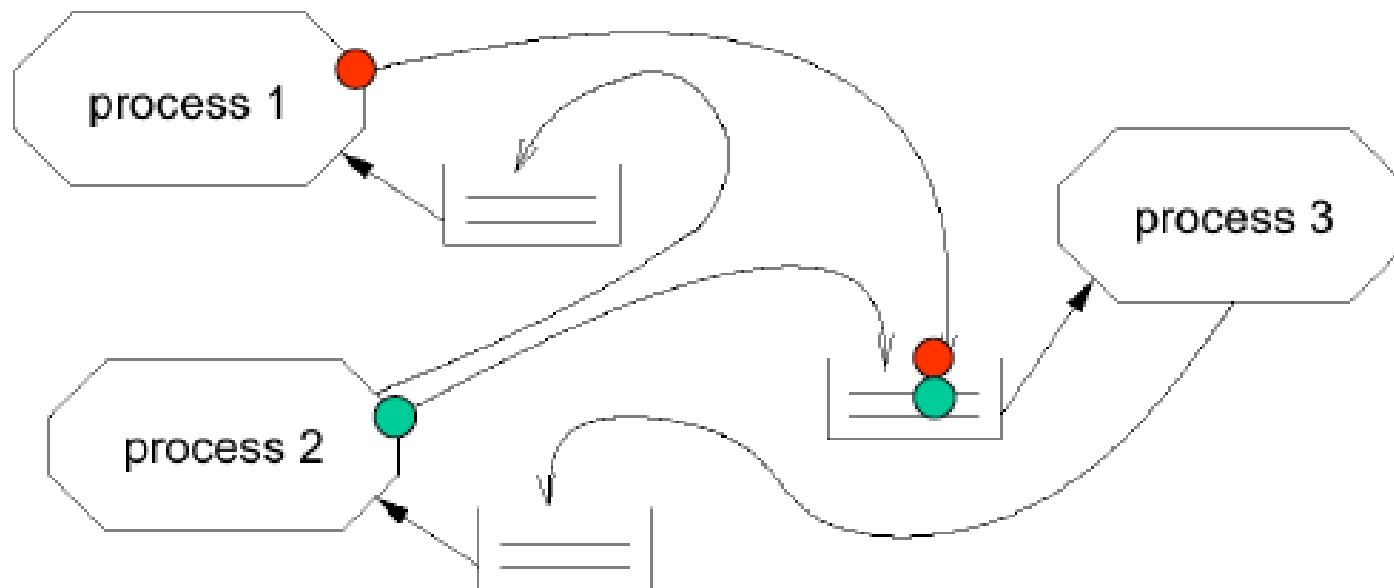


1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.4 SDL

→ **Deterministic** ? All orders are legal: simulators can show different behaviors for the **same input**, all of which are correct.



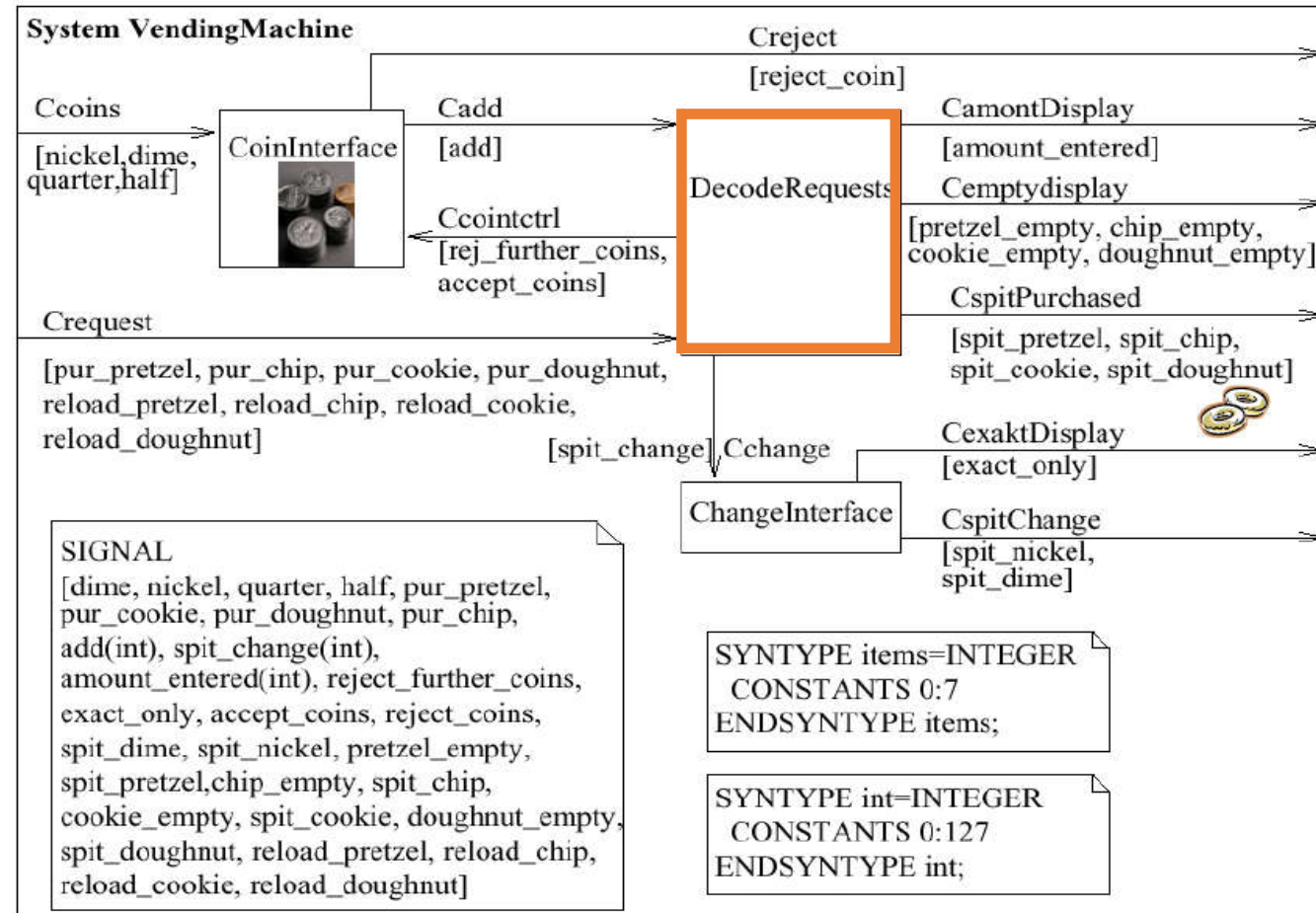
1. NGÔN NGỮ LẬP TRÌNH

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1.2 Specification languages

1.2.4 SDL

Vending machine



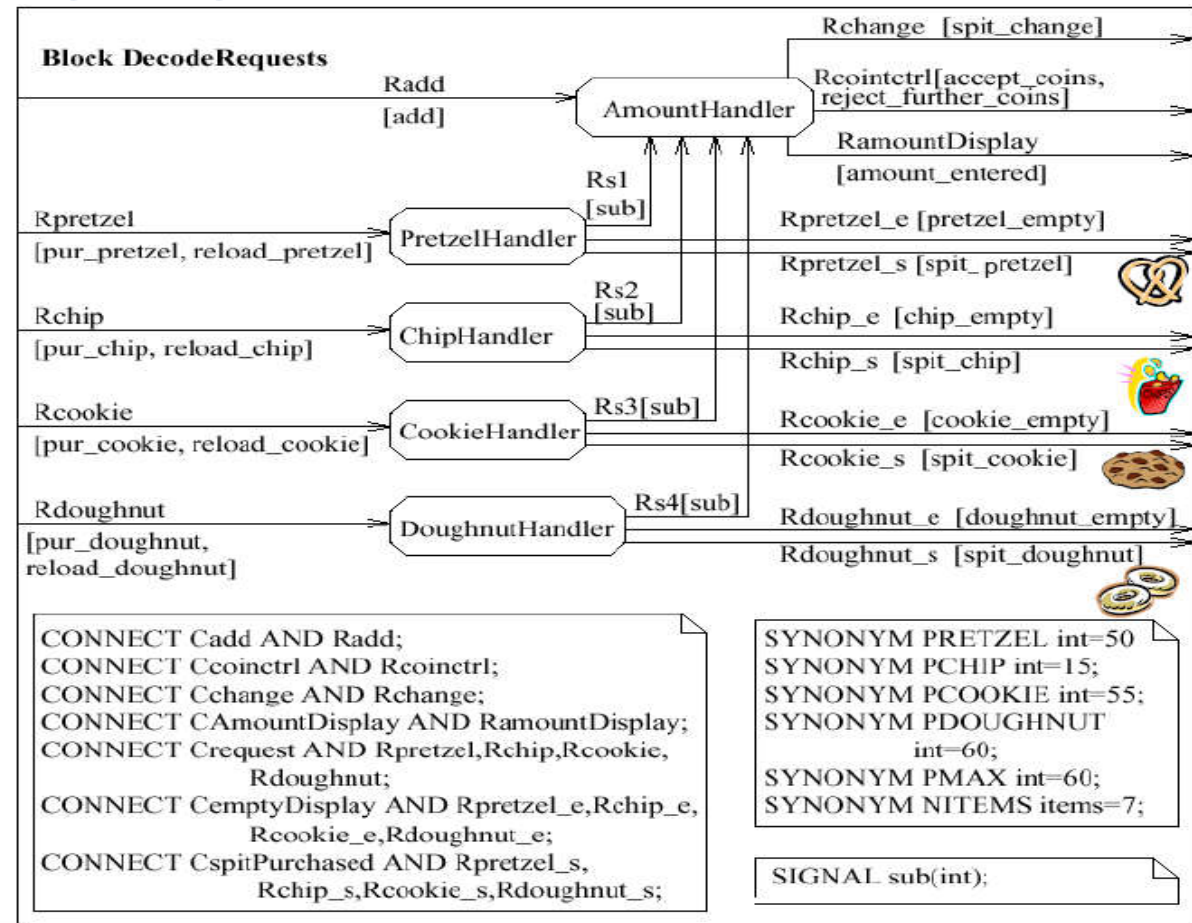
1. NGÔN NGỮ LẬP TRÌNH

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1.2 Specification languages

1.2.4 SDL

Decode Requests



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- Statemate system
- SDL
- **SystemC**
- SpecC
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.5 SystemC

Requirements and solutions for **modeling Hardware** in **a Software language**:

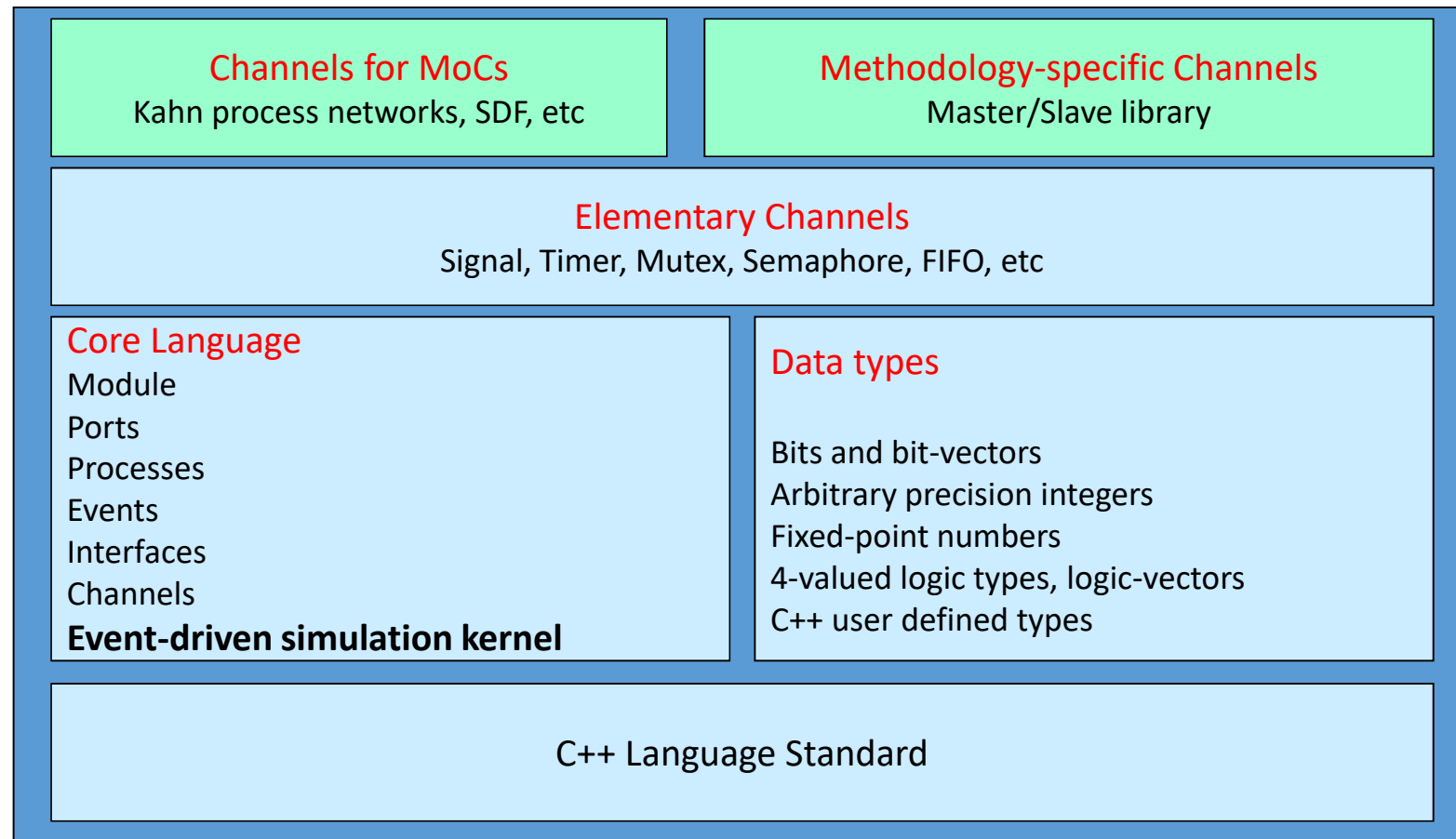
- **C++ class library** including required functions.
- **Concurrency**: via processes, controlled by sensitivity lists and calls to wait primitives.
- **Time**: Floating point numbers in SystemC 1.0 and Integer values in SystemC 2.0; Units ps, ns, μ s,...
- **Support of bit-datatypes**: bitvectors of different lengths; multiple-valued logic (2 and 4; resolution*)
- **Communication**: plug-and-play channel model, allowing easy replacement of intellectual property.
- **Deterministic behavior** not guaranteed.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.5 SystemC

→ Architecture



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- Statemate system
- SDL
- SystemC
- **SpecC**
- VHDL, Verilog, SystemVerilog
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

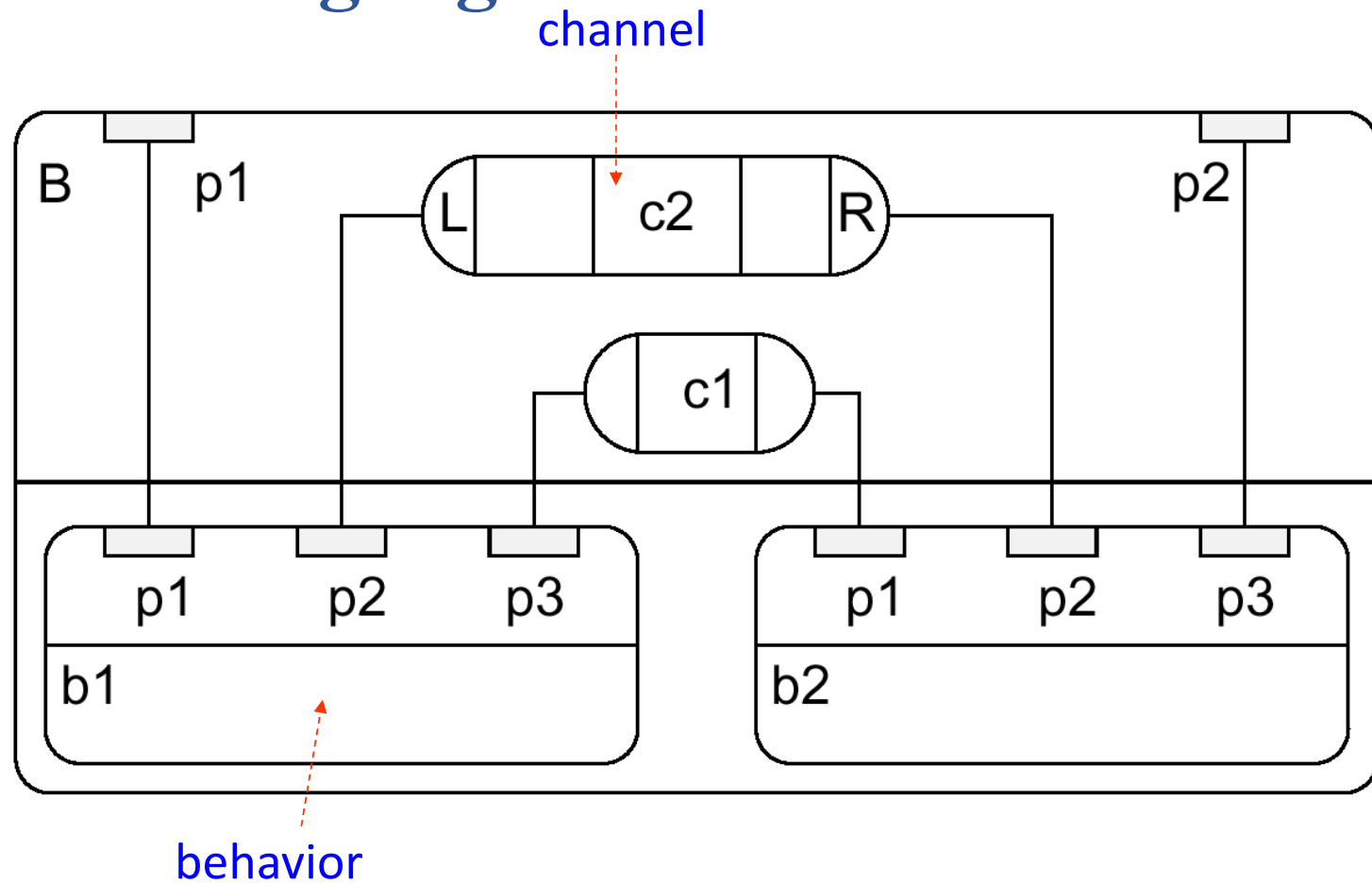
1.2.6 SpecC

- SpecC is based on the **clear separation** between **communication** and **computation**.
 - **Enables plug-and-play** for system components;
 - **Models** system as hierarchical networks of behaviors communicating through channels.
- **SpecC specifications** consists of behaviors, channels and interfaces.
 - **Behaviors** include ports, locally instantiated components, private variables and functions and a public main function.
 - **Channels** encapsulate communication. They include variables and functions, used for the definition of a communication protocol.
 - **Interfaces** are linking behaviors and channels together. They declare the communication protocols which are defined in a channel.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.6 SpecC



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

- Harel's StateCharts
- UML Statecharts
- Statemate system
- SDL
- SystemC
- SpecC
- **VHDL, Verilog, SystemVerilog**
- Simulink
- C, C++, Java.

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.8 VHDL

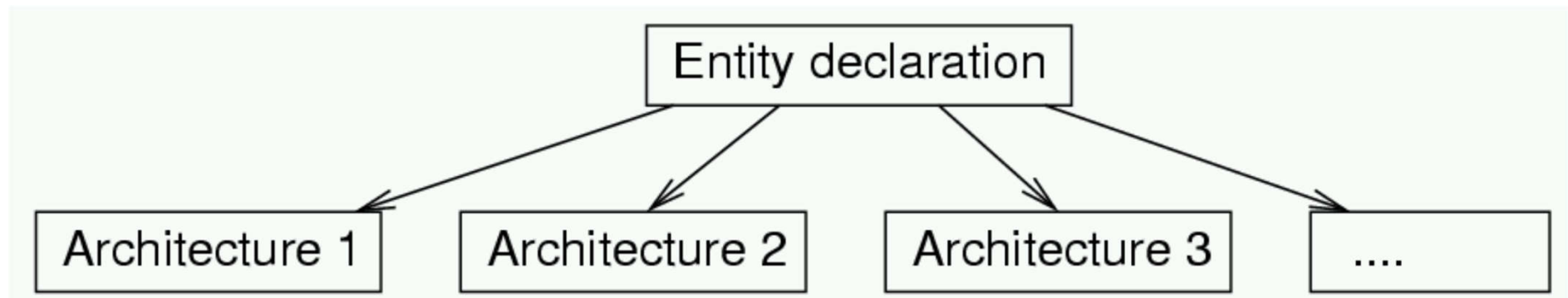
- **HDL** = Hardware Description Language
- Textual HDLs replaced graphical HDLs in the 1980s (better for complex behavior).
- 1980: Definition started by DoD in 1980
- 1984: first version of the language defined, based on ADA, PASCAL
- 1987: IEEE standard 1076; 1992 revision;
- Recently: VHDL-AMS models analog

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.8 VHDL – Entities and Architectures

- Each design unit is called an entity.
- Entities are comprised of entity declarations.
- Each architecture includes a model of the entity. By default, the most recently analyzed architecture is used. The use of another architecture can be requested in a configuration one or several architectures.



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.8 VHDL → Entity Declaration

```
entity full_adder:
```

```
    port(a, b, carry_in: in Bit;    -- input ports
```

```
          sum,carry_out: out Bit); --output ports
```

```
end full_adder;
```



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.8 VHDL → Architecture

architecture behavior of full_adder is

```
begin
  sum      <= (a xor b) xor carry_in  after 10 Ns;
  carry_out <= (a and b) or (a and carry_in) or (b and carry_in) after 10 Ns;
end behavior;
```



1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.8 VHDL - Summary

- Behavioral hierarchy (procedures & functions)
- Structural hierarchy but no nested processes
- No object-orientation
- Static number of processes
- Complicated simulation semantics
- Too low level for initial specification
- Good for intermediate language for hardware generation

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.9 Verilog

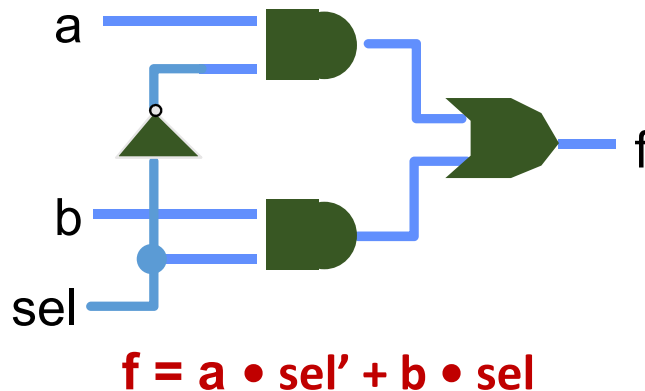
- HW description language competing with VHDL
- Standardized: IEEE 1364-1995 (V1.0) and IEEE 1364-2001 (V2.0)
- Features similar to VHDL:
 - Designs described as connected entities
 - Bit-vectors and time units are supported
- Features that are different:
 - Built-in support for 4-value logic and for logic with 8 strength levels encoded in two bytes per signal.
 - More features for transistor-level descriptions
- Less flexible than VHDL
- More popular in the US (VHDL common in Europe)

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.9 Verilog – Structural Models

- Built from gate primitives **and/or** other modules.
- They describe the circuit using logic gates – much as we would see in an implementation of a circuit.



```
module mux
  (output f,
   input  a, b, sel);

  and #5 g1 (f1, a, nsel),
    g2 (f2, b, sel);

  or  #5 g3 (f, f1, f2);
  not g4 (nsel, sel);
end module
```

1. NGÔN NGỮ LẬP TRÌNH

1.2 Specification languages

1.2.10 Other languages (environments)

- **SCE**: is a toolset for modeling, synthesis and validation of multi-processor embedded system designs
- **SPARK**: C-to-VHDL high-level synthesis framework
- **SpecCharts**: Combination of StateCharts and VHDL
- **MATLAB** (Matrix Laboratory): facility for defining matrix-based computations, extending numerical FORTRAN packages LINPACK and EISPACK with a GUI
- **Simulink**: GUI-based specification of control systems, uses MATLAB for solving these problems
- **Esterel**: reactive language; synchronous; all reactions are assumed to be in 0 time; communication based on instantaneous broadcast.